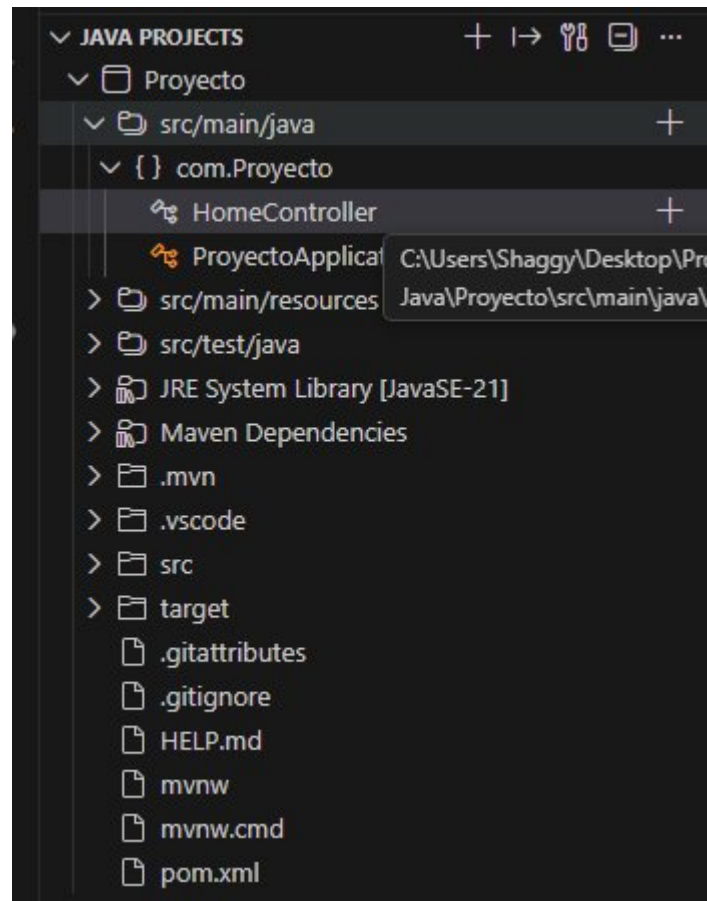


1.0) Inicializacion de proyecto (test para comprobar que todo este correcto):



- Creamos el archivo HomeController en la ruta : com.P-HomeContCreamsrc -> main -> java -> com.Proyecto.

1.1) Y al controlador le agregamos el siguiente contenido:

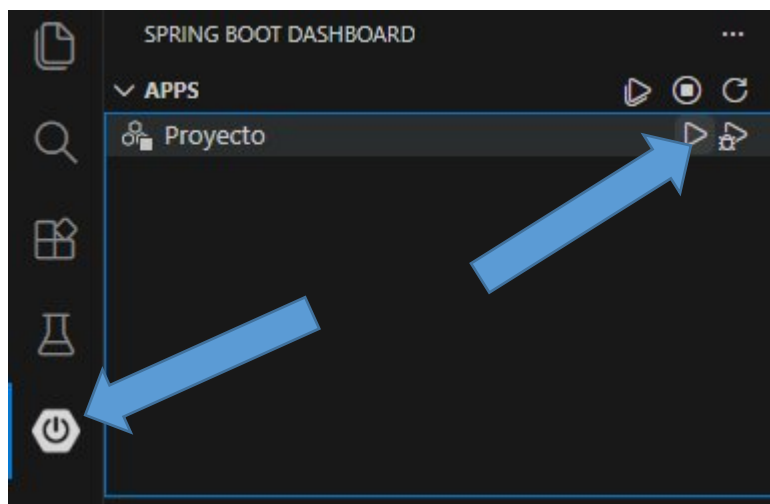
```
//Importar paquete exacto segun la ruta en el proyecto
package com.Proyecto;

//Importacion de librerias de Spring Boot.
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;

//Componente encargado de manejar las solicitudes HTTP.
@Controller
//Creacion de la clase HomeController (se debe llamar igual que el archivo creado).
public class HomeController{
    //Mapear las solicitudes HTTP Get.
    @GetMapping("/home")
    //Creacion de la funcion del tipo cadena de caracteres.
    public String Home(){
        //Devuelve en la pagina la palabra home.
        return "home";
    }
}
```

- El **@Controller** es importante, porque es el que le dice a nuestro controlador que este es una clase del tipo web que maneja solicitudes **HTTP**.
- El **@GetMapping** es importante porque es el que maneja o administra las solicitudes **HTTP** del tipo **Get**.

1.2) Correr el servidor:



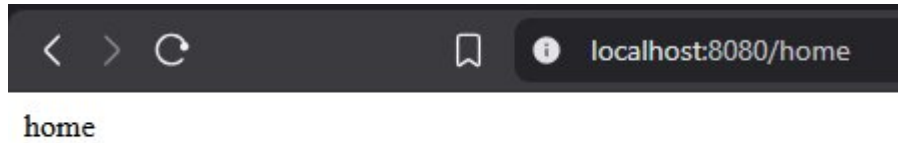
- Esto inicializara el servidor, que por defecto corre en el puerto 8080.

```
C:\Users\Shaggy\Desktop\Projectos Java\Projectos & C:\Program Files\Microsoft\jdk-21.0.10-hotspot\bin/java.exe -gc:(Users\Shaggy\AppData\Local\Temp)\cp_cg0lupzpkd3328a3omy6co0
7,argfile" com.Projecto.ProyectoApplication
```

```
( ) _ _ _ _ _ O _ _ _ _ _ \ / \ / \ / \
W _ _ [ D | I | L | F | G ] _ _ _ ) ) ) )
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
:: Spring Boot ::                (v4.0.2)
```

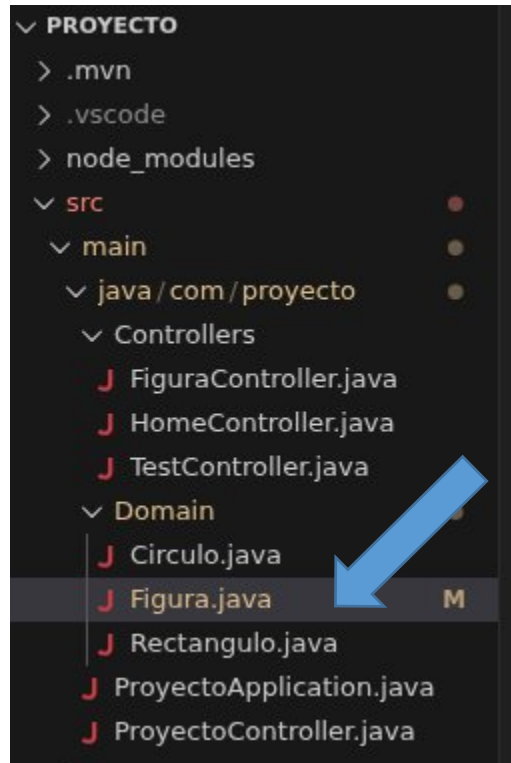
```
2026-02-16T11:15:08.180-05:00 INFO 3216 --- [Projecto] [    main] com.Projecto.ProyectoApplication : Starting ProyectoApplication using Java 21.0.10 with PID 3216 (C:\Users\Shaggy\Desktop\Projectos Java
ycto\target\classes started by Shaggy in C:\Users\Shaggy\Desktop\Projectos Java\Projecto)
2026-02-16T11:15:08.192-05:00 INFO 3216 --- [Projecto] [    main] com.Projecto.ProyectoApplication : No active profile set, falling back to 1 default profile: "default"
2026-02-16T11:15:09.841-05:00 INFO 3216 --- [Projecto] [    main] o.s.boot.tomcat.TomcatWebServer : Tomcat initialized with port 8080 (http)
2026-02-16T11:15:09.867-05:00 INFO 3216 --- [Projecto] [    main] o.apache.catalina.core.StandardService : Starting service [/Tomcat]
2026-02-16T11:15:09.870-05:00 INFO 3216 --- [Projecto] [    main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/11.0.15]
2026-02-16T11:15:09.961-05:00 INFO 3216 --- [Projecto] [    main] b.w.c.s.WebApplicationContextInitializer : Root WebApplicationContext: initialization completed in 1634 ms
2026-02-16T11:15:10.712-05:00 INFO 3216 --- [Projecto] [    main] o.s.boot.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
2026-02-16T11:15:10.723-05:00 INFO 3216 --- [Projecto] [    main] com.Projecto.ProyectoApplication : Started ProyectoApplication in 3.538 seconds (process running for 4.775)
```

1.3) Verificamos que el servidor se este ejecutando correctamente:



- Abrimos nuestro navegador de preferencia y escribimos **localhost:8080/home**
- Y este nos devolvera lo que contenia **return**.

1.4) Creacion de la clase padre **Figura:**



- Creamos la carpeta Domain dentro de : **src -> main -> java -> com.proyecto.**

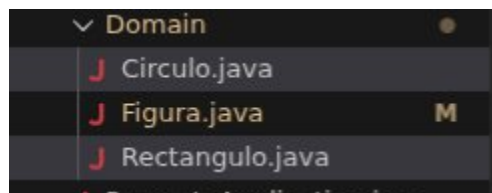
- Y dentro de la carpeta Domain creamos el archivo **Figura.java**.

1.5) Agregamos la estructura de la clase padre **Figura**:

```
src > main > java > com > proyecto > Domain > J Figura.java
1 | //Importar paquete exacto segun la ruta en el proyecto.
2 | package com.proyecto.Domain;
3 |
4 | //Creacion de la clase Figura
5 | public abstract class Figura {
6 |     //Creacion de la variable cadena de caracteres String el cual guarda el nombre del objeto.
7 |     private final String Nombre;
8 |
9 |     // Creacion de la funcion Figura
10 |    protected Figura(String Nombre) {
11 |        //Comprobacion del contenido Nombre, si la variable Nombre devuelve Nulo o vacio.
12 |        if (Nombre == null || Nombre.isBlank()){
13 |            //Si la condicion se cumple, muestra el mensaje de error.
14 |            throw new IllegalArgumentException ("El nombre no puede ser nulo o vacio");
15 |        }
16 |
17 |        //Guarda el cotenido de Nombre del parametro en la variabble Nombre del objeto
18 |        this.Nombre = Nombre;
19 |    }
20 |    //Creacion de un get para obtener (retornar) la variable Nombre
21 |    public String getNombre() {
22 |        //Retorno de la variable Nombre
23 |        return Nombre;
24 |    }
25 |    //Funcion que permite hacer el calculo del area para las figuras
26 |    public abstract double calcularArea();
27 | }
```

- En la línea **2** colocamos la ruta exacta del controlador.
- Y la línea **26** es nuestra funcion para los objetos que crearemos mas adelante los cuales usaran polimorfismo ya que van a imprimir informacion diferente en pantalla.

1.6) Creacion de los archivos Circulo y Rectangulo:



- Es importante colocarle la primera letra en **MAYUSCULAS** para que la momento de crear la clase este no de problemas en la ejecucion del codigo.

1.7) Agregamos la siguiente estructura a los 2 archivo recientemente creados (Circulo y Rectangulo) Herando de la clase padre **Figura**:

src > main > java > com > proyecto > Domain > J Circulo.java

```
1 | //Importar paquete exacto segun la ruta del proyecto.
2 | package com.proyecto.Domain;
3 |
4 | //Creacion del objeto Circulo que Hereda de Figura (Herencia).
5 | public class Circulo extends Figura{
6 |     //Creacion de variable para el radio.
7 |     private final double radio;
8 |
9 |     //Creacion de definicion del constructor (Constructor).
10 |    public Circulo(double radio){
11 |        //Creacion del objeto (Instancia)
12 |        super("circulo");
13 |        //Comprobacion de la variable radio.
14 |        if (radio <= 0) {
15 |            //Si la condicion se cumple muestra mensaje de error.
16 |            throw new IllegalArgumentException("El radio debe ser mayor que cero");
17 |        }
18 |        //Guardar valor del parametro en el atributo del bojeto.
19 |        this.radio = radio;
20 |    }
21 |    //Creacion de la funcion Get para obtener el valor del atributo radio del objeto.
22 |    public double GetRadio(){
23 |        //Retonar lo que contiene radio.
24 |        return radio;
25 |    }
26 |
27 |    @Override
28 |    //Llama a la funcion para calcular el area de la figura (Poliformismo).
29 |    public double calcularArea(){
30 |        //Retonar el valor de ratio multiplicando PI * radio.
31 |        return Math.PI * radio;
32 |    }
33 | }
```

```

src > main > java > com > proyecto > Domain > J Rectangulo.java
1 | //Importar paquete exacto segun la ruta en el proyecto.
2 | package com.proyecto.Domain;
3 |
4 | //Creacion de la clase Rectangulo haciendo herencia a Figura.
5 | public class Rectangulo extends Figura {
6 |     //Creacion de las variables base y altura.
7 |     private final double base;
8 |     private final double altura;
9 |
10 |    //Creacion del constructor
11 |    public Rectangulo(double base, double altura){
12 |        //Inicializacion del objeto
13 |        super("Rectangulo");
14 |        //Comprobacion de base o altura si son mayor a 0
15 |        if (base <= 0 || altura <= 0){
16 |            //Mensaje de error si la condicion se cumple.
17 |            throw new IllegalArgumentException("La base o altura debe ser mayor a 0");
18 |        }
19 |        //Guardado del contenido de los parametros en la variables (inicio de las variables)
20 |        this.base = base;
21 |        this.altura = altura;
22 |    }
23 |
24 |    //Creacion de un get para la base.
25 |    public double GetBase(){
26 |        //Retorna base.
27 |        return base;
28 |    }
29 |
30 |    //Creacion de un get para la altura.
31 |    public double GetAltura(){
32 |        //Retorna la altura.
33 |        return altura;
34 |    }
35 |
36 |    //Sirve para indicar explicitamente que un metodo hijo esta reemplazando o implementando
37 |    // un metodo de su clase padre
38 |    @Override
39 |
40 |    //Llamado de la funcion del objeto padre para calcular el area del objeto (Polimorfismo)
41 |    public double calcularArea(){
42 |        return base * altura;
43 |    }
44 | }
45 |

```

- Como vemos en las imagenes de los 2 objetos, la funcion **calcularArea** viene **heredada** de la clase padre **Figura** y aqui aplicamos **polimorfismo** ya que la funcion en cada objeto **imprime valores diferente en pantalla**.

1.8) Creacion del Controlador FiguraController.java:

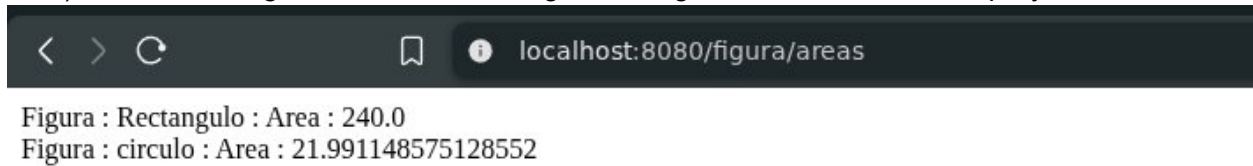
```
src > main > java > com > proyecto > Controllers > J FiguraController.java
1  //Importar paquete exacto segun la ruta del proyecto
2  package com.proyecto.Controllers;
3  //Importacion de librerias.
4  import java.util.ArrayList;
5  import java.util.List;
6  import org.springframework.web.bind.annotation.GetMapping;
7  import org.springframework.web.bind.annotation.RequestMapping;
8  import org.springframework.web.bind.annotation.RestController;
9
10 //Importacion de los archivos recientemente creados segun su ruta.
11 import com.proyecto.Domain.Circulo;
12 import com.proyecto.Domain.Figura;
13 import com.proyecto.Domain.Rectangulo;
14
15 //Se utiliza para crear controladores de servicios webs.
16 @RestController
17
18 //Sirve para vincular solicitudes HTTP a metodos dentro del controlador.
19 @RequestMapping("/figura")
20 //Creacion de la clase para el controlador.
21 public class FiguraController {
22     //Sirve para vincular solicitudes HTTP GET a metodos dentro del controlador.
23     @GetMapping("/areas")
24     //Creacion de la funcion que calcula las areas.
25     public String GetAreas(){
26         //Creacion de una lista que accede a la memoria RAM (vector en C++)
27         // el cual accede dinamicamente a la memoria ram
28         List<Figura> figura = new ArrayList<>();
29         //Instacia de los objetos (Crear) Rectangulo y Circulo con sus respectivas areas.
30         figura.add(new Rectangulo(12, 20));
31         figura.add(new Circulo(7));
32
33         //Creacion de variable del tipo de cadena de caracteres vacia la cual
34         // contendra los valores de los calculos.
35         String resultado = "";
36         //Creacion de un bucle (for each) que itera en la lista Figura y guarda cada iteracion en la variable figura
37         for(Figura f : figura){
38             //Concatenacion de los resultados.
39             resultado += "Figura : " + f.getNombre() + " : " + "Area : " + f.calcularArea() + "<br>";
40         }
41         //Retorno de los resultados.
42         return resultado;
43     }
44 }
45
```

- Este es el que controlara y mostrara en pantalla los valores de los objetos **Circulo** y **Rectangulo** que heredan de la clase padre **Figura** los cuales usan **Polimorfismo** para imprimir en pantalla diferentes valores.

1.9) Ahora ejecutamos el servidor web:

- Por favor volver a ver la imagen en el articulo 1.2.

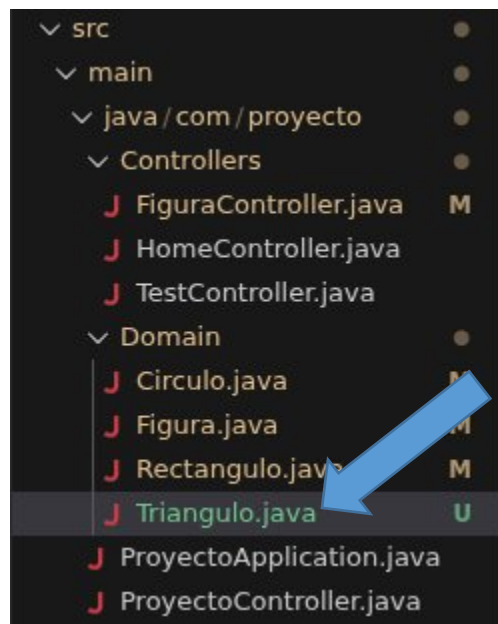
1.10) Ahora nos dirigimos a nuestro navegador e ingresamos en la ruta del proyecto:



- Aquí entramos en la ruta **localhost:8080/figura/areas**, así mostrando en pantalla del return el valor de las áreas de los objetos los cuales vemos en pantalla.

Ejercicio de POO en Spring Boot, Creación de un objeto del tipo Triangulo

2.0) Creación del archivo Triangulo.java:



- El archivo .java debe estar dentro de la ruta: **src -> main -> java -> com.proyecto -> Domain**.

2.1) Contenido del objeto Triangulo:

```
src > main > java > com > proyecto > Domain > J Triangulo.java > Triangulo > calcularArea()
1 //Importar paquete exacto segun la ruta del proyecto.
2 package com.proyecto.Domain;
3
4 //Creacion de la clase Triangulo que hereda de Figura.
5 public class Triangulo extends Figura{
6     //Creacion de las variables base y altura.
7     private double base;
8     private double altura;
9     public Triangulo(double base, double altura){
10         //Constructor del Triangulo
11         super(Nombre: "Triangulo");
12         //Comprobacion de base y altura sean mayor a 0.
13         if(base >= 0 || altura >= 0){
14             this.base = base;
15             this.altura = altura;
16         }
17         //De lo contrario mostrara mensaje de error.
18         else {
19             throw new IllegalArgumentException("Error la base o altura deben ser mayor a 0");
20         }
21     }
22
23     //Creacion del Get que retornara base.
24     public double GetBase(){
25         return base;
26     }
27
28     //Creacion del Get que retornara altura.
29     public double GetAltura(){
30         return altura;
31     }
32
33     //Sirve para indicar explicitamente que un metodo hijo esta reemplazando o implementando
34     //un metodo de su clase padre
35     @Override
36
37     //Llamado de la funcion del objeto padre para calcular el area del objeto (Polimorfismo)
38     public double calcularArea(){
39         return (base * altura) / 2;
40     }
41 }
42
```

- El objeto **Triangulo** esta **heredando** de la clase padre **Figura**
- Y esta usando **polimorfismo** para calcular el area llamado la funcion **calcularArea()**.

2.2) Modificación del archivo FiguraController para instanciar el objeto Triangulo:

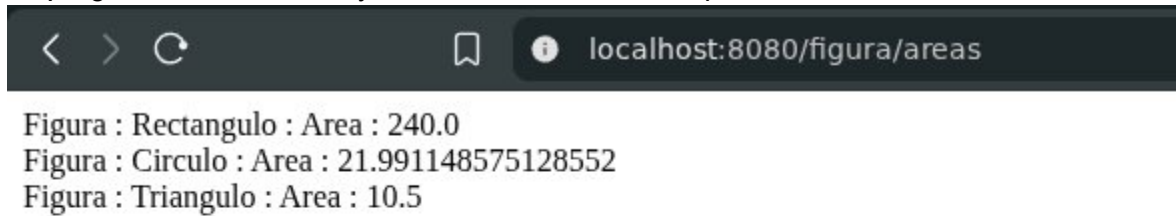
```
src > main > java > com > proyecto > Controllers > FiguraController.java > Language Support for Java(TM) by Red Hat > % FiguraController > GetAreas()
1 | //Importar paquete exacto segun la ruta del proyecto
2 | package com.proyecto.Controllers;
3 | //Importacion de librerias.
4 | import java.util.ArrayList;
5 | import java.util.List;
6 | import org.springframework.web.bind.annotation.GetMapping;
7 | import org.springframework.web.bind.annotation.RequestMapping;
8 | import org.springframework.web.bind.annotation.RestController;
9 |
10 | //Importacion de los archivos recientemente creados segun su ruta.
11 | import com.proyecto.Domain.Circulo;
12 | import com.proyecto.Domain.Figura;
13 | import com.proyecto.Domain.Rectangulo;
14 |
15 | //Importamos el triangulo.
16 | import com.proyecto.Domain.Triangulo;
17 |
18 | //Se utiliza para crear controladores de servicios webs.
19 | @RestController
20 |
21 | //Sirve para vincular solicitudes HTTP a metodos dentro del controlador.
22 | @RequestMapping("/figura")
23 | //Creacion de la clase para el controlador.
24 | public class FiguraController {
25 |     //Sirve para vincular solicitudes HTTP GET a metodos dentro del controlador.
26 |     @GetMapping("/areas")
27 |     //Creacion de la funcion que calcula las areas.
28 |     public String GetAreas(){
29 |         //Creacion de una lista que accede a la memoria RAM (vector en C++)
30 |         // el cual accede dinamicamente a la memoria ram
31 |         List<Figura> figura = new ArrayList<>();
32 |         //Instacia de los objetos (Crear) Rectangulo y Circulo con sus respectivas areas.
33 |         figura.add(new Rectangulo(base: 12, altura: 20));
34 |         figura.add(new Circulo(radio: 7));
35 |         //Agregamos a la lista el objeto Triangulo.
36 |         figura.add(new Triangulo(base: 7, altura: 3));
37 |         //Creacion de variable del tipo de cadena de caracteres vacia la cual
38 |         // contendra los valores de los calculos.
39 |         String resultado = "";
40 |         //Cracion de un bucle (for each) que itera en la lista Figura y guarda cada iteracion en la variable figura
41 |         for(Figura f : figura){
42 |             //Concatenacion de los resultados.
43 |             resultado += "Figura : " + f.getNombre() + " : " + "Area : " + f.calcularArea() + "<br>";
44 |         }
45 |         //Retorno de los resultados.
46 |         return resultado;
47 |     }
48 | }
```

- Aqui vemos que importamos el objeto **Triangulo** y lo agregamos a la lista figura para que despues se pueda **instanciar** el objeto.
- Para que el **for each** recorra los objetos e **imprima sus valores en pantalla**.

2.3) Correr el servidor:

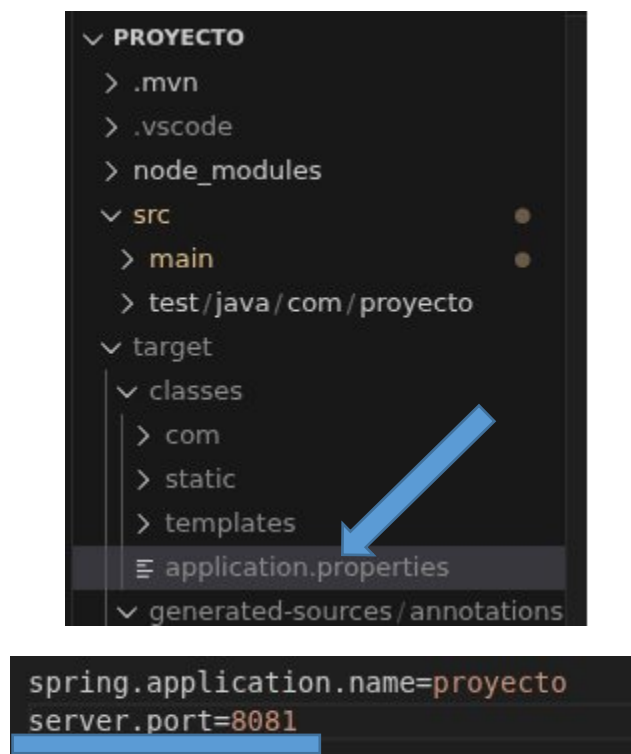
- Por favor volver a ver la imagen en el articulo **1.2**.

2.4) Ingresamos en la ruta y vemos los resultados en pantalla:



- Como vemos en la imagen, el objeto se **instancion** correctamente y esta **retornando** el valor de la funcion **calcularArea()**.

3.0) Cambiar puerto del servidor:

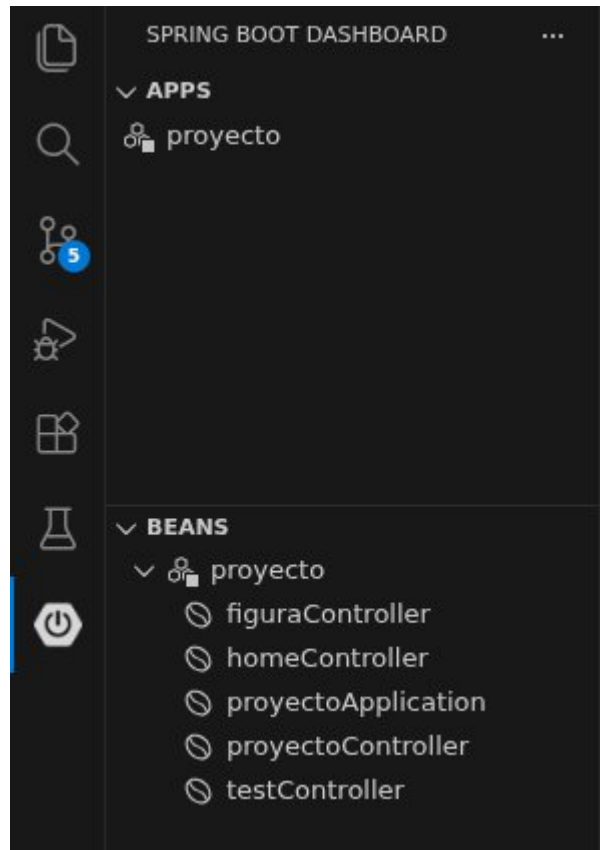


- Muchas veces tenemos que cambiar el puerto del proyecto, ya que puede darse la casualidad de que el puerto que usa por defecto de **Spring Boot** el cual es el **8080** esta

siendo escuchado por otro programa o inclusive por **otro servidor web** y es necesario cambiar el puerto ya que sin eso el **servidor no se ejecutara**.

- Abrimos el archivo **application.properties** que en mi caso se encuentra en la ruta: **target -> classes -> application.properties**.
- Y agregamos la linea **server.port=8081** (en mi caso elegi el 8081 ya que este puerto no lo estoy utilizando).

3.1) Ejecutar servidor con el puerto cambiado y entramos en la nueva ruta:



(Imagen donde se resalta el cambio de puerto).

```

  :: Spring Boot ::                (v4.0.2)

2026-02-16T12:24:27.272-05:00 INFO 69889 --- [projectos] [ restartedMain] com.projectos.ProjectosApplication : Starting ProjectosApplication using Java 21.0.10 with PID 69889 (/home/shaggy/Downloads/proyectos)
2026-02-16T12:24:27.285-05:00 INFO 69889 --- [projectos] [ restartedMain] com.projectos.ProjectosApplication : No active profile set, falling back to 1 default profile: 'default'
2026-02-16T12:24:27.576-05:00 INFO 69889 --- [projectos] [ restartedMain] .e.DevToolsProperty.DefaultsPostProcessor : Devtools property defaults active! Set 'spring.devtools.add-properties' to 'false' to disable
2026-02-16T12:24:27.576-05:00 INFO 69889 --- [projectos] [ restartedMain] .e.DevToolsProperty.DefaultsPostProcessor : For additional web related logging consider setting the 'logging.level.web' property to 'DEBUG'
2026-02-16T12:24:30.773-05:00 INFO 69889 --- [projectos] [ restartedMain] o.s.b.o.t.c.TomcatWebServer : Tomcat initialized with port 8881 (http)
2026-02-16T12:24:30.832-05:00 INFO 69889 --- [projectos] [ restartedMain] o.a.p.c.c.s.StandardService : Starting service [Tomcat]
2026-02-16T12:24:30.834-05:00 INFO 69889 --- [projectos] [ restartedMain] o.a.p.c.c.s.StandardEngine : Starting Servlet engine: [Apache Tomcat/11.0.15]
2026-02-16T12:24:30.941-05:00 INFO 69889 --- [projectos] [ restartedMain] b.w.c.s.WebApplicationContextInitializer : Root WebApplicationContext: initialization completed in 3358 ms
2026-02-16T12:24:32.395-05:00 INFO 69889 --- [projectos] [ restartedMain] o.s.b.o.t.c.TomcatWebServer : Tomcat started on port 8881 (http) with context path '/'
2026-02-16T12:24:32.420-05:00 INFO 69889 --- [projectos] [ restartedMain] org.springframework.boot : Started ProjectosApplication in 6.494 seconds (process running for 8.787)

```



- Como vemos en la image, ingresando a la nueva ruta : **localhost:8081/figura/areas**, este entra correctamente, esto quiere decir que el cambio de puerto a sido **exitoso**.

Respuesta a las preguntas y glosario de terminos.

4.0) Respuesta a lo que se encuentra en la pagina 14 en la guia: GuíasegundaclasesenaPOO (1).pdf.

Los 4 pilares de **POO** (Programacion Orientada a Objetos).

- 1) **Encapsulamiento**: Es definir el acceso a los **atributos** (variables) y **metodos** (funciones) del objeto de como seran accedidos dependiendo si son : **publicas**, **privadas** o **protegidas**.
- 2) **Herencia**: Permite crear una clase **hija** reutilizando **metodos** y **atributos** de la clase **padre**.
- 3) **Abstraccion**: Permite definir correctamente los **atributos** (variables) y **metodos** (funciones) ignorando detalles **innecesarios**.
- 4) **Polimorfismo**: Permite llamar el metodo de la clase **padre** y poder **modificar** su **comportamiento** para poder **cambiar su valor**.

Glosario

Termino	Definicion
Clase	Plantilla que defines los atributos y metodos.
Encapsulamiento	Ocultar funciones criticas y solo dejar publicas las operaciones seguras.
Herencia	Reutilizar metodos y atributos de la clase padre.
Abstraccion	Definir metodos y atributos necesarrios para la clase.
Polimorfismo	Cambiar la respuesta del metodo de la clase padre.